PUC MINAS

Engenharia de Software
Unidade Lurdes

Detecção de Bad Smells e Refatoração Segura

Análise, refatoração e validação de um gerador de relatórios JavaScript

Disciplina: Teste de Software

Aluno: Victor Gabriel Cruz Pereira

Matrícula: 861270

Projeto: bad-smells-js-refactoring
Data: 1 de novembro de 2026

Fork: github.com/Victorgabrielcruz/bad-smells-js-refactoring

Relatório elaborado a partir do código preservado e das evidências reais das fases de análise e validação.

1. Diagnóstico do código original

Os três smells abaixo foram identificados em **src/ReportGenerator.js**, no método **generateReport**. A inspeção manual e a análise estática são complementares.

Método longo e responsabilidades misturadas | linhas 11-68

O método monta cabeçalho, seleciona itens, altera prioridade, formata linhas, soma valores e fecha o relatório. Essa concentração amplia o impacto de mudanças e obriga os testes a preparar e verificar o relatório completo para exercitar regras isoladas. Técnica: **Extract Method**, com auxiliares internos ao módulo.

Condicionais complexas | linhas 27-54, especialmente 43-52

Papel do usuário, limite de valor e formato são decididos em ramos aninhados. A manutenção exige conferir combinações ADMIN/USER, CSV/HTML e valores de fronteira. Técnica: **Decompose Conditional** e cláusula de guarda, separando visibilidade e renderização.

Duplicação de código | linhas 35-37 e 46-48

A linha CSV é repetida nos ramos ADMIN e USER; a soma também aparece nas linhas 37, 41, 48 e 51. Alterações podem divergir entre caminhos, exigindo testes redundantes para detectar inconsistências. Técnica: extrair a geração da linha e centralizar a totalização.

Resultado real do ESLint antes da refatoração

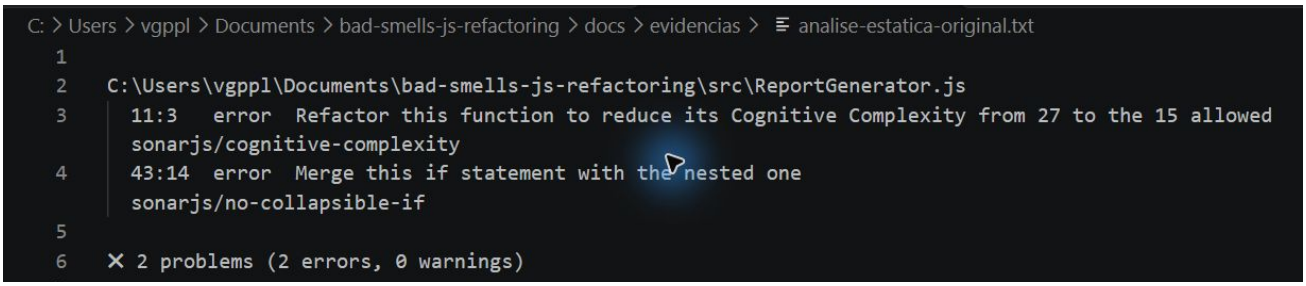


Figura 1. Captura real da saída de npx eslint src/ preservada em analise-estatica-original.txt, aberta no editor nesta entrega. Recorte de enquadramento; não é uma captura histórica do terminal.

O **SonarJS** quantificou a complexidade cognitiva em **27** (limite **15**, linha 11:3) e apontou uma condicional unificável (43:14). Acrescentou medida e localização precisa aos achados manuais. Não emitiu diagnóstico de funções idênticas: blocos CSV repetidos não equivalem a funções inteiras iguais.

2. Refatoração do problema prioritário

A prioridade foi decompor **generateReport**, que concentrava as decisões e tarefas. O arquivo original foi preservado; a implementação está em **src/ReportGenerator.refactored.js**, com o mesmo export, construtor, propriedade db e assinatura pública.

Antes | original, linhas 43-54

```
} else if (user.role === 'USER') {
  // Users comuns só veem itens de valor baixo
  if (item.value <= 500) {
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  }
}
```

Depois | refatorado, linhas 14-33

```
generateReport(reportType, user, items) {
  let report = createHeader(reportType, user);
  let total = 0;

  for (const item of items) {
    if (!isVisibleTo(user, item)) {
      continue;
    }

    applyAdminPriority(user, item);
    if (reportType === 'CSV' || reportType === 'HTML') {
      report += createRow(reportType, user, item);
      total += item.value;
    }
  }

  report += createFooter(reportType, total);

  return report.trim();
}
```

Extract Method: createHeader, createFooter e createRow isolam a formatação; isVisibleTo nomeia a regra de acesso; applyAdminPriority mantém o efeito sobre o item. São funções internas, sem hierarquia ou novos métodos públicos.

Decompose Conditional e guard clause: itens não visíveis são descartados no início do laço. **Consolidate Duplicate Conditional Fragments:** CSV e soma deixam de aparecer em cada ramo. **Replace Magic Number with Symbolic Constant:** 1000 e 500 recebem os nomes ADMIN_PRIORITY_THRESHOLD e USER_MAX_VISIBLE_VALUE, mantendo os operadores originais.

Preservações relevantes: prioridade preexistente, mutação de item.priority, formatos desconhecidos e saída HTML ADMIN com <tr >. O espaço não foi corrigido porque sua remoção alteraria o texto retornado.

3. Validação e rede de segurança

ESLint 8.57.1 e SonarJS 0.21.0 foram executados com a mesma configuração efetiva para as duas versões. Os presets recomendados e as regras explícitas permaneceram ativos: complexidade com limite 15 e funções idênticas, ambos com severidade error. O JSON é compatível com ESLint 8; essa versão está fora de manutenção, opção registrada para compatibilidade com a base.

Verificação	Original	Refatorado
ESLint	2 erros; 0 avisos	0 erros; 0 avisos
Complexidade cognitiva	27 (limite 15)	Sem violação > 15*
Condicional unificável	1 ocorrência	Nenhuma
Testes com ESM temporário	4 passam; 1 falha	4 passam; 1 falha
npm test sem opção ESM	Falha de carregamento	Falha de carregamento

* O lint não informou a complexidade exata da cópia por não haver violação. Ausência de alerta não significa complexidade zero.

O que foi comprovado

O diff em relação ao HEAD está vazio para src/ReportGenerator.js e tests/ReportGenerator.test.js; os hashes coincidem com a base registrada. A suíte real está em tests/, embora o enunciado cite __tests__/. A cópia foi criada em __tests__/ReportGenerator.refactored.test.js e difere integralmente da original apenas no import.

Após cada mudança significativa, os testes foram executados com ESM temporário. Resultado final real: **8 aprovados e 2 reprovados em 10 testes**. A mesma falha de HTML ADMIN ocorre em cada suíte: espera-se <tr>, mas recebe-se <tr >. A base já apresentava essa divergência. Nenhuma expectativa ou regra foi relaxada.

Comparação suplementar entre versões: **200 cenários aprovados**, com duas chamadas por cenário e 400 comparações de saída e estado. Inclui limites 500/1000, prioridades preexistentes, formatos/papéis desconhecidos e itens vazios. Não constitui prova para toda entrada JavaScript possível.

Conclusão e limitação da entrega

Os testes funcionam como rede de segurança quando existe uma linha de base conhecida e se verifica o comportamento após pequenas mudanças. Aqui, permitiram distinguir a falha preexistente de regressões novas, não identificadas nos casos executados. A decomposição reduz o custo de compreender e alterar as regras, e o lint confirmou a remoção dos dois alertas na cópia.

A exigência de 100% dos testes aprovados não foi atingida. npm test também requer a opção temporária --experimental-vm-modules para carregar ESM. Essas pendências foram mantidas explícitas; a preservação do comportamento não permite alegar aprovação completa.

Rastreabilidade: docs/analise-manual.md; docs/processo-refatoracao.md; docs/evidencias/analise-estatica-original.eslint.json; validacao-final-eslint.json; validacao-final-jest.json e validacao-final.md. Enunciado: PDF fornecido pelo aluno.